Image Transmission Between Two Smartphones Using Acoustic Binary FSK

Digital modulation project with BER analysis (MATLAB)

Name: _____ **Roll No:** _____ **Exp. No:** _____ **Date:** _____

1. Objective

To send a digital colour image from one smartphone to another **through sound** using Binary Frequency Shift Keying (BFSK), to recover the image at the receiver using MATLAB, and to measure the Bit Error Rate (BER) of the link.

Additional studies (simulation): effect of distance and noise, error-control coding, DBPSK compared with FSK, and verification of the theoretical error probability of non-coherent FSK.

Requirements: MATLAB on a PC, two smartphones (one plays the audio, the other records it), image file.

2. Block Diagram

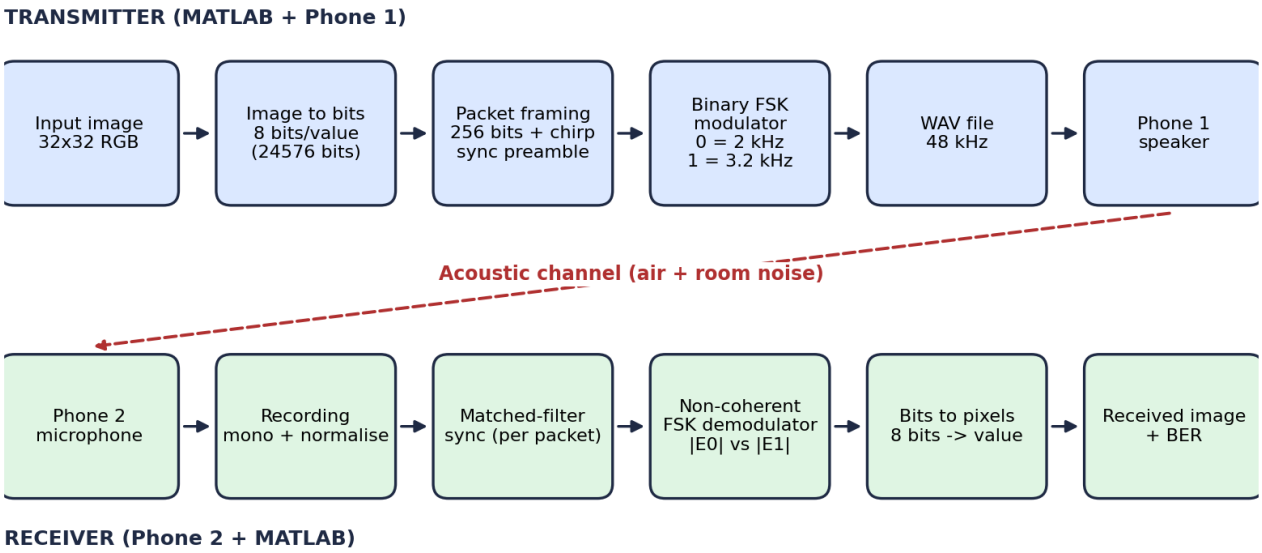


Fig. 1: Block diagram of the acoustic FSK image transmission system

Working

- The image is resized to 32 x 32 and each colour value (R, G, B; 0 to 255) is converted to 8 bits, giving 24576 bits.
- The bits are divided into 96 packets of 256 bits. A short chirp (1 kHz to 4 kHz, 0.1 s) is placed in front of every packet for synchronisation.
- FSK modulation: bit **0** is sent as a 2000 Hz tone and bit **1** as a 3200 Hz tone, each lasting 2.5 ms (400 bits/s). The result is saved as *fsk_image.wav*.
- Phone 1 plays the WAV file; Phone 2 records it. The air between them is the channel.

- In MATLAB the receiver finds each chirp with a matched filter, then for every 2.5 ms slot compares the energy at 2000 Hz and 3200 Hz. The larger one gives the bit.
- Bits are grouped in 8s to get pixel values and the image is rebuilt. It is compared with the original to calculate the BER.

Parameter	Value	Parameter	Value
Sampling frequency	48 kHz	Packet size	256 bits
Bit rate	400 bits/s	Sync chirp	1 to 4 kHz, 0.1 s
Tone for bit 0 (f0)	2000 Hz	Image size	32 x 32 x 3 (RGB)
Tone for bit 1 (f1)	3200 Hz	Total bits	24576
Samples per bit	120	Number of packets	96

3. MATLAB Codes

Keep the image, both programs and the recording in the same MATLAB folder. Run *fsk_tx.m* first, then *fsk_rx.m* after recording.

3.1 Transmitter: fsk_tx.m

```
%% FSK IMAGE TRANSMITTER  (Image -> Bits -> FSK -> WAV)
clc; clear; close all;

%% ---------------- Parameters (must match fsk_rx.m) ----------------
Fs      = 48000;          % audio sampling rate (Hz)
Rb      = 400;            % bit rate (bits/sec)
f0      = 2000;           % tone for bit 0 (Hz)
f1      = 3200;           % tone for bit 1 (Hz)
sps     = Fs/Rb;          % samples per bit (120)

% Image resolution:
% [32 32] RGB -> ~61 seconds audio (recommended for quick phone testing)
% [64 64] RGB -> ~245 seconds audio (higher detail, longer playback)
imgSize = [32 32];
pktBits = 256;            % bits per packet
Tc      = 0.1;            % sync chirp length (sec)
fa = 1000; fb = 4000;     % chirp start / end frequency (Hz)

%% ---------------- Image -> bits ----------------
% Check for available image file (prioritizes image.jpg, then image.png, then lion.jpg)
if exist('image.jpg', 'file')
    imgName = 'image.jpg';
elseif exist('image.png', 'file')
    imgName = 'image.png';
elseif exist('lion.jpg', 'file')
    imgName = 'lion.jpg';
else
    error('Image file not found! Place image.jpg (or image.png / lion.jpg) in the working directory.');
end
imgRaw = imread(imgName);

% Keep full RGB color channels if available
rIdx = round(linspace(1, size(imgRaw, 1), imgSize(1)));
cIdx = round(linspace(1, size(imgRaw, 2), imgSize(2)));

if size(imgRaw, 3) >= 3
    isColor = true;
    img = uint8(imgRaw(rIdx, cIdx, 1:3));
    fprintf('Loaded COLOR image: %s (%dx%d RGB)\n', imgName, imgSize(1), imgSize(2));
else
    isColor = false;
    img = uint8(imgRaw(rIdx, cIdx));
    fprintf('Loaded GRAYSCALE image: %s (%dx%d)\n', imgName, imgSize(1), imgSize(2));
end

% Serialize pixels to binary stream (8 bits per color component, MSB first)
bits = reshape((dec2bin(img(:), 8) - '0').', [], 1);
nBits = numel(bits);

% Pad so bits fit into whole packets
pad = mod(-nBits, pktBits);
bitsPadded = [bits; zeros(pad,1)];
nPkt = numel(bitsPadded)/pktBits;

%% ---------------- Sync chirp (start of every packet) ----------------
tc = (0:round(Tc*Fs)-1)'/Fs;
chirp = sin(2*pi*(fa*tc + (fb-fa)/(2*Tc)*tc.^2));
N = numel(chirp);
chirp = chirp .* (0.5*(1-cos(2*pi*(0:N-1)'/(N-1))));    % smooth window

%% ---------------- FSK modulation ----------------
tb = (0:sps-1)'/Fs;
s0 = sin(2*pi*f0*tb);
s1 = sin(2*pi*f1*tb);

x = zeros(0,1);
for p = 1:nPkt
    b = bitsPadded((p-1)*pktBits + (1:pktBits)).';      % 1 x pktBits
    w = s0*(1-b) + s1*b;                                % sps x pktBits
    x = [x; chirp; w(:)];                               %#ok<AGROW>
end
x = [zeros(Fs,1); x; zeros(Fs,1)];                      % 1 s silence at both ends
x = 0.9 * x / max(abs(x));

audiowrite('fsk_image.wav', x, Fs);
save('txdata.mat', 'bits', 'img', 'imgSize', 'isColor'); % used by receiver for BER and color display

fprintf('Bits: %d | Packets: %d | Duration: %.1f sec\n', nBits, nPkt, numel(x)/Fs);
fprintf('Saved fsk_image.wav -> play it on Phone 1.\n');

%% ---------------- Plots ----------------
figure;
subplot(2,2,1); imshow(img); title('Original image');
subplot(2,2,2); plot((0:3*sps*8-1)/Fs*1000, x(Fs+numel(chirp)+(1:3*sps*8)));
xlabel('ms'); title('FSK waveform (data part)');
subplot(2,2,[3 4]); spectrogram(x, 1024, 512, 1024, Fs, 'yaxis');
title('Spectrogram of transmitted signal'); ylim([0 6]);
```

3.2 Receiver: fsk_rx.m

```
%% FSK IMAGE RECEIVER  (Recording -> Sync -> FSK demod -> Image)
clc; clear; close all;

%% ---------------- Parameters (must match fsk_tx.m) ----------------
Fs      = 48000;
Rb      = 400;
f0      = 2000;
f1      = 3200;
sps     = Fs/Rb;
imgSize = [64 64];
pktBits = 256;
Tc      = 0.1;
fa = 1000; fb = 4000;

recFile = 'received.mp3';     % <-- your phone recording (.m4a, .wav, .mp3)
if ~exist(recFile, 'file')
    if exist('received.wav', 'file')
        recFile = 'received.wav';
    elseif exist('fsk_image.wav', 'file')
        fprintf('"%s" not found. Using local "fsk_image.wav" for test/loopback.\n', recFile);
        recFile = 'fsk_image.wav';
    else
        error('Recording file "%s" not found! Place your phone recording in this folder.', recFile);
    end
end

%% ---------------- Load recording ----------------
[y, Fr] = audioread(recFile);
y = mean(y, 2);                         % to mono
if Fr ~= Fs, y = resample(y, Fs, Fr); end
y = y - mean(y);
y = y / max(abs(y));

%% ---------------- Rebuild the sync chirp ----------------
tc = (0:round(Tc*Fs)-1)'/Fs;
chirp = sin(2*pi*(fa*tc + (fb-fa)/(2*Tc)*tc.^2));
N = numel(chirp);
chirp = chirp .* (0.5*(1-cos(2*pi*(0:N-1)'/(N-1))));

% matched filter: output at index n = chirp ending at sample n
c = abs(filter(flipud(chirp), 1, y));

%% ---------------- Find first packet ----------------
first = find(c > 0.5*max(c), 1);
[~, off] = max(c(first : min(first+N, numel(c))));
chirpEnd = first + off - 1;

%% ---------------- Total packets & frame sizing ----------------
if exist('txdata.mat', 'file')
    txInfo = load('txdata.mat', 'imgSize', 'isColor', 'bits');
    imgSize = txInfo.imgSize;
    if isfield(txInfo, 'isColor')
        isColor = txInfo.isColor;
    else
        isColor = false;
    end
    nBitsTotal = numel(txInfo.bits);
else
    isColor = true; % default assumption
    nBitsTotal = imgSize(1) * imgSize(2) * 3 * 8;
end
nPkt = ceil(nBitsTotal/pktBits);
pktLen = N + pktBits*sps;               % samples per packet (nominal)

%% ---------------- Demodulate packet by packet ----------------
tb = (0:sps-1)';
E0 = exp(-1j*2*pi*f0*tb/Fs);
E1 = exp(-1j*2*pi*f1*tb/Fs);

rxBits = zeros(nPkt*pktBits, 1);
W = 300;                                % sync search window (samples) for clock drift

for p = 1:nPkt
    if p > 1
        expected = chirpEnd + pktLen;
        lo = max(expected - W, 1);
        hi = min(expected + W, numel(c));
        [~, o] = max(c(lo:hi));
        chirpEnd = lo + o - 1;
    end
    startIdx = chirpEnd + 1;
    stopIdx  = startIdx + pktBits*sps - 1;
    if stopIdx > numel(y)
        warning('Recording ended early at packet %d of %d.', p, nPkt);
        break;
    end
    seg = reshape(y(startIdx:stopIdx), sps, pktBits);
    e0 = abs(E0.' * seg);               % energy at f0
    e1 = abs(E1.' * seg);               % energy at f1
```

```
    rxBits((p-1)*pktBits + (1:pktBits)) = (e1 > e0).';
end
rxBits = rxBits(1:nBitsTotal);

%% ---------------- Bits -> image ----------------
b = reshape(rxBits, 8, []).';
pix = uint8(b * (2.^(7:-1:0))');

if isColor
    imgRx = reshape(pix(1:imgSize(1)*imgSize(2)*3), [imgSize(1), imgSize(2), 3]);
else
    imgRx = reshape(pix(1:imgSize(1)*imgSize(2)), imgSize);
end

%% ---------------- BER (if tx data is available) & Plots ----------------
figure('Name', 'FSK Receiver Results', 'NumberTitle', 'off');
if exist('txdata.mat', 'file')
    load('txdata.mat', 'bits', 'img');
    nErr = sum(bits ~= rxBits);
    BER = nErr / numel(bits);
    fprintf('Bit errors: %d / %d   |   BER = %.4f (%.2f%%)\n', nErr, numel(bits), BER, BER*100);

    subplot(2,2,1); imshow(img);   title('Original Transmitted Image');
    subplot(2,2,2); imshow(imgRx); title(sprintf('Received Image (BER = %.4f)', BER));

    % Error map: mean difference across color channels
    if isColor
        errMap = uint8(mean(abs(double(img) - double(imgRx)), 3));
    else
        errMap = uint8(abs(double(img) - double(imgRx)));
    end
    subplot(2,2,3); imshow(errMap, []); title('Error Map (Differences)');
    subplot(2,2,4); plot(c); title('Sync Correlation Output'); xlabel('Sample Index'); ylabel('Magnitude'); grid on;
else
    subplot(1,2,1); imshow(imgRx); title('Received Image');
    subplot(1,2,2); plot(c); title('Sync Correlation Output'); xlabel('Sample Index'); ylabel('Magnitude'); grid on;
end
imwrite(imgRx, 'received_image.png');
```

3.3 Extension programs (simulation)

File	Purpose	Output figure
extension_a_distance_noise.m	BER versus distance and noise level	extension_a_results.png
extension_b_channel_coding.m	Uncoded vs Repetition(3) vs Hamming(7,4)	extension_b_results.png
extension_c_dbpsk.m	DBPSK compared with FSK (BER, PSNR, bandwidth)	extension_c_results.png
extension_d_sim_ber.m	Monte-Carlo BER vs Eb/N0 against theory	extension_d_results.png

4. Observations

4.1 Input (transmitter side)

Item	Observed value
Input image file	image.png, resized to 32 x 32, RGB colour
Number of bits	32 x 32 x 3 x 8 = 24576 bits
Number of packets	24576 / 256 = 96
Time per packet	0.1 s (chirp) + 0.64 s (256 bits) = 0.74 s
Length of fsk_image.wav	about 73 s (71 s of data + 1 s silence at each end)

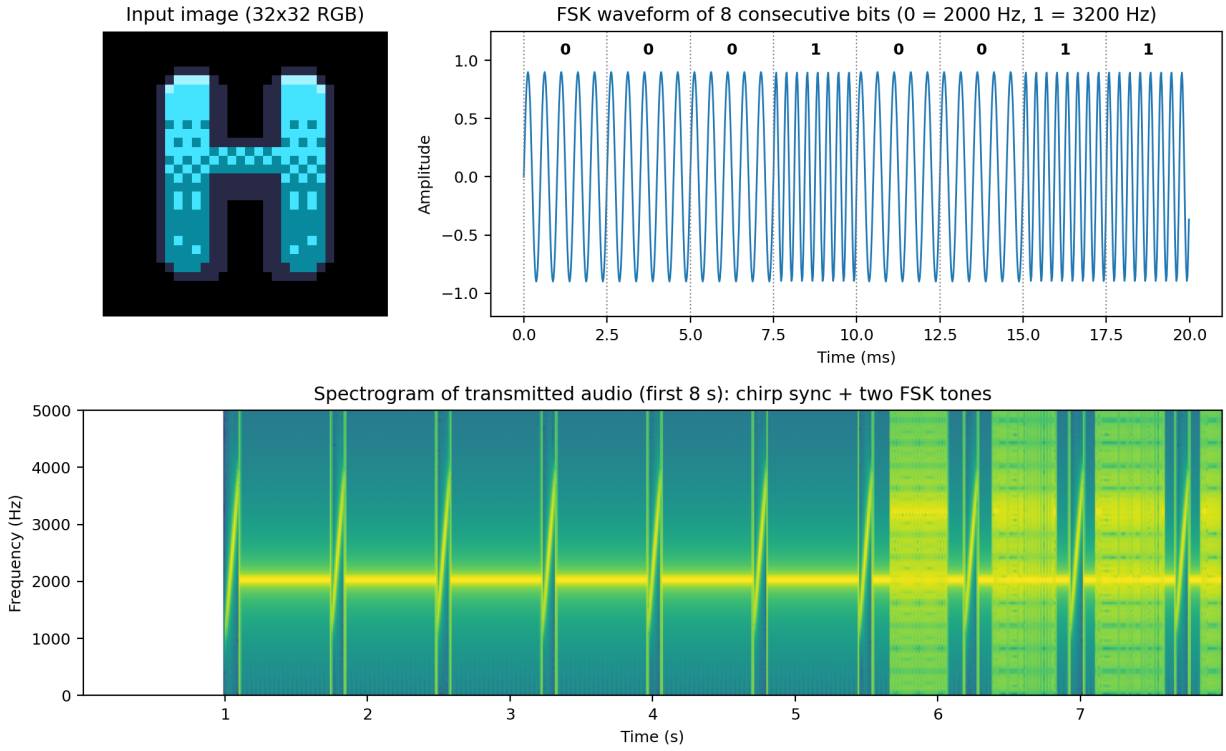


Fig. 2: Input image, FSK waveform of 8 bits (low tone = 0, high tone = 1) and spectrogram of the transmitted audio. The slanted lines are the sync chirps; the horizontal bands are the 2000 Hz and 3200 Hz tones.

4.2 Output (receiver side, real phone-to-phone test)

The WAV file was played on one phone and recorded on the other (received.mp3, about 74 s). The receiver detected all 96 sync peaks, one for each packet.

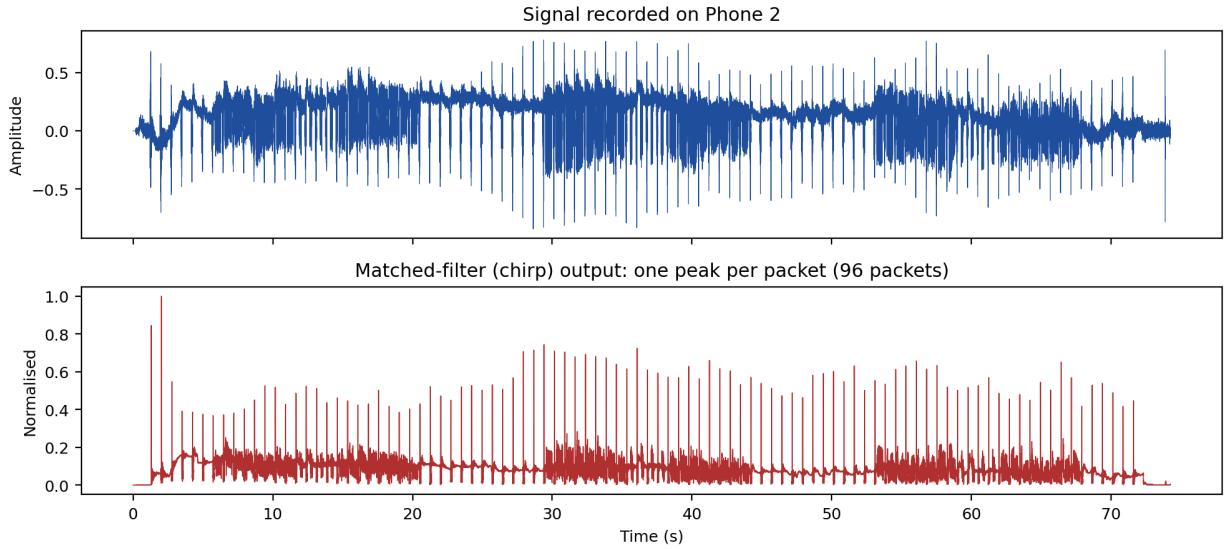


Fig. 3: Recording on Phone 2 and matched-filter output (one peak per packet)

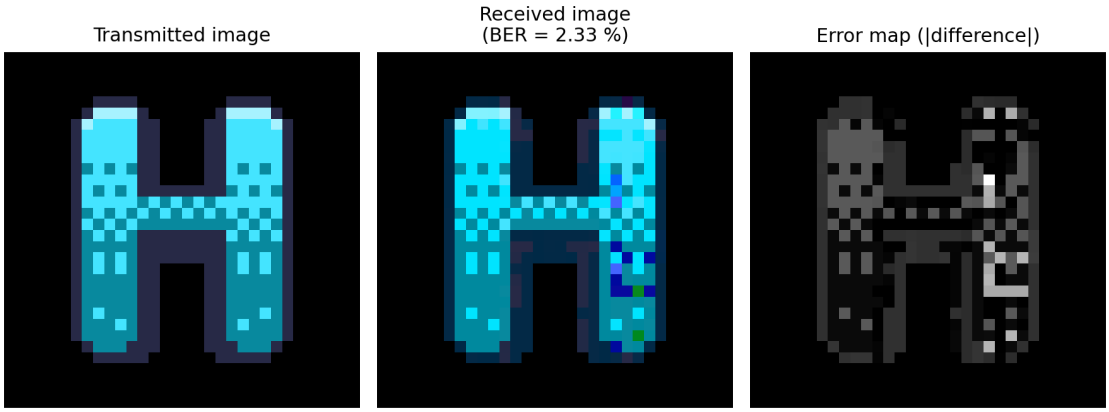


Fig. 4: Transmitted image, received image and error map

Item	Observed value
Total bits transmitted	24576
Bit errors	572
Bit Error Rate (BER)	572 / 24576 = 0.0233 (2.33 %)
Packets received without error	71 out of 96
Pixel values with at least one wrong bit	329 out of 1024 pixels
PSNR of received image	24.2 dB

- The received image is clearly recognisable and has the same shape and colours as the original.
- Errors occurred in bursts inside 25 packets (up to 51 errors in one packet); the other 71 packets were error free.
- A wrong bit in a low-order position only changes the shade slightly, while a wrong high-order bit gives a visibly wrong colour (blue/green dots in Fig. 4).

4.3 Simulation observations (Extensions A to D)

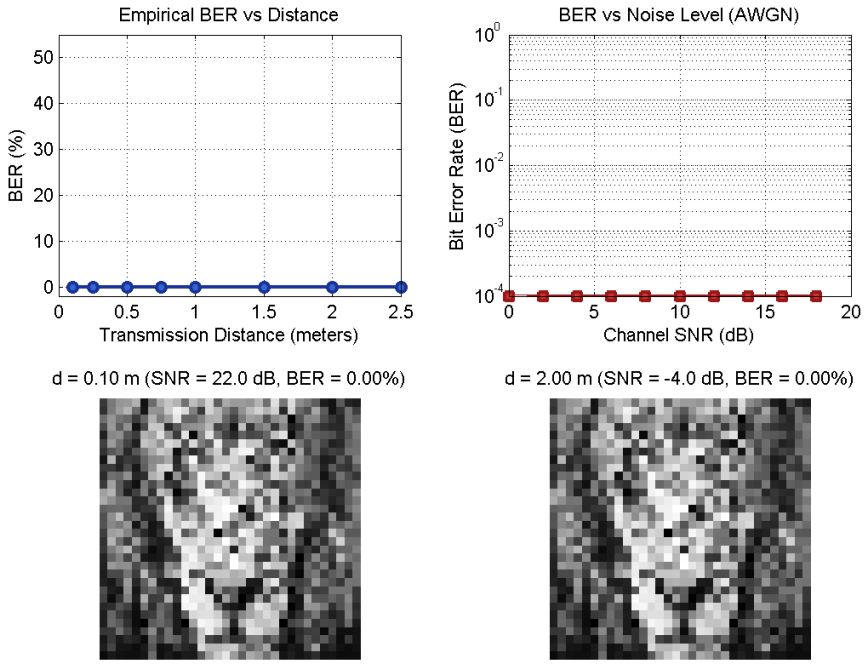


Fig. 5: Extension A - BER versus distance and noise level

Observation: BER was 0 % for all distances from 0.1 m to 2.5 m (simulated with 1/d sound pressure loss and AWGN), and the image was received perfectly even at 2 m (SNR = -4 dB). This is because the receiver adds up 120 samples per bit, so Eb/N0 is about 17.8 dB higher than the channel SNR. The simulation does not include room echo, so the real test (Section 4.2) is more demanding.

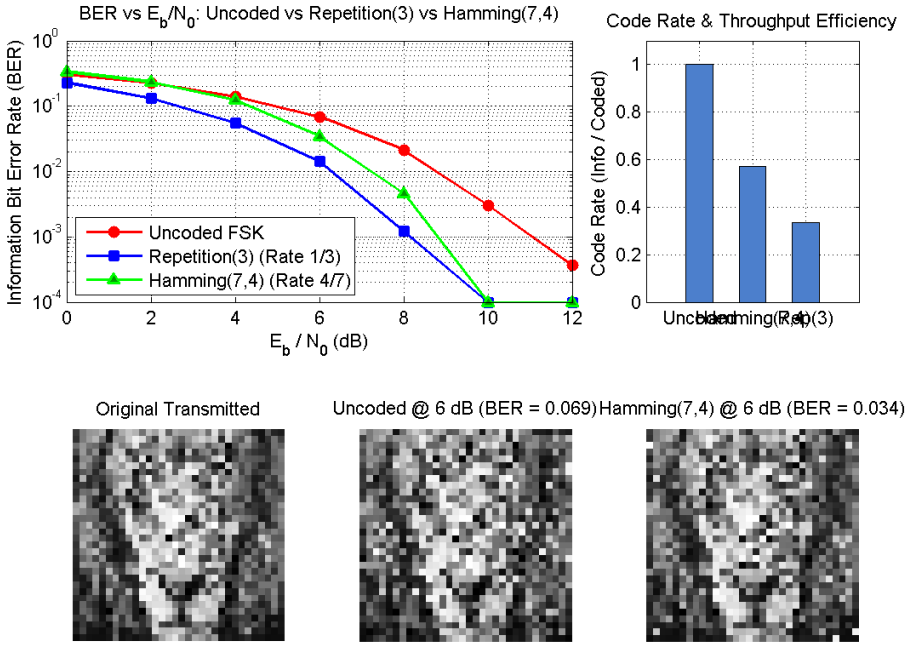


Fig. 6: Extension B - Uncoded FSK vs Repetition(3) vs Hamming(7,4)

Observation: At Eb/N0 = 6 dB the BER was 6.9 % (uncoded), 3.4 % (Hamming 7,4) and about 1.4 % (Repetition 3). At 10 dB both coded schemes gave no errors. The price is a lower code rate: 4/7 for Hamming and 1/3 for repetition, so the transmission takes longer.

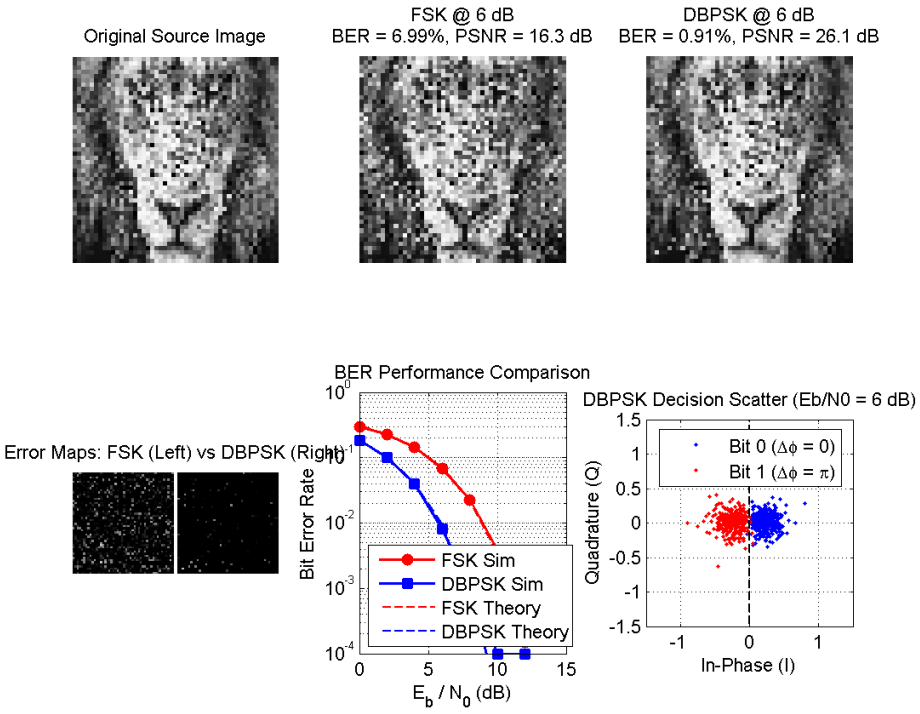


Fig. 7: Extension C - FSK compared with DBPSK at Eb/N0 = 6 dB

Observation: At 6 dB, FSK gave BER = 6.99 % (PSNR 16.3 dB) and DBPSK gave BER = 0.91 % (PSNR 26.1 dB). DBPSK uses a single carrier and about half the bandwidth of FSK for the same bit rate.

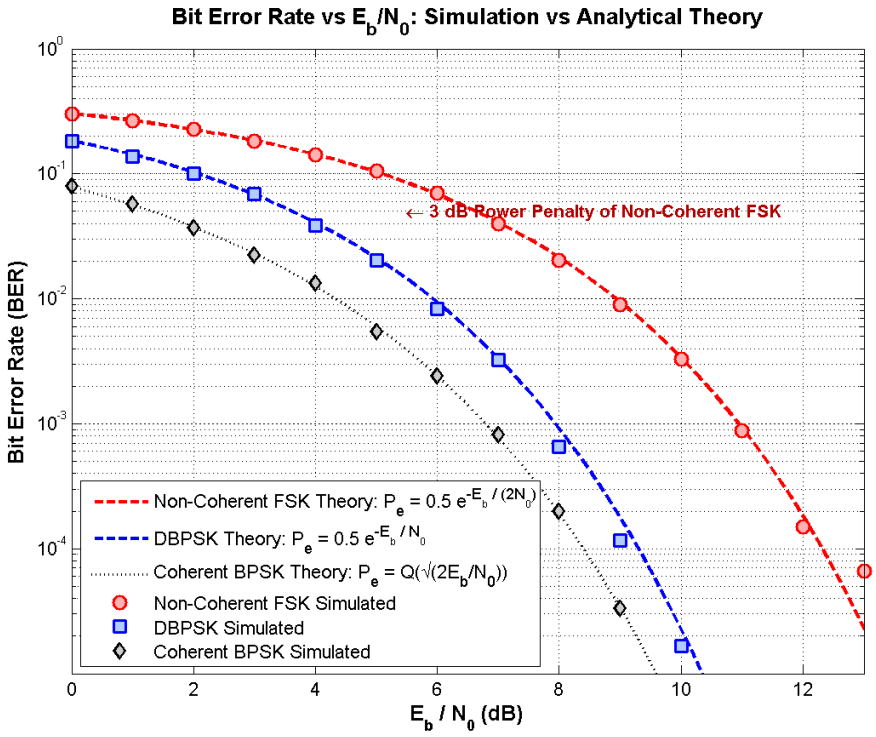


Fig. 8: Extension D - simulated BER compared with theoretical curves

Observation: The simulated points follow the theoretical curves Pe = 0.5 exp(-Eb/2N0) for non-coherent FSK and Pe = 0.5 exp(-Eb/N0) for DBPSK. DBPSK needs about 3 dB less Eb/N0 than non-coherent FSK for the same BER.

5. Conclusion

- A colour image was successfully sent from one smartphone to another as sound using Binary FSK, and was recovered in MATLAB with a BER of 2.33 %.
- Non-coherent FSK needs no carrier phase information, so it works well with the unknown delay and distortion of a phone speaker and microphone.
- The chirp placed before every packet kept the receiver synchronised for all 96 packets, even though the two phones have different clocks.
- The errors came in bursts, which shows that the real room is noisier than the simple AWGN model. Simulation showed that Hamming or Repetition coding reduces the BER, at the cost of speed.
- Simulation also confirmed the theory: DBPSK is about 3 dB more power efficient than non-coherent FSK, but it was not tested over the air in this experiment.
- The main limitation is speed: at 400 bits/s even a 32 x 32 colour image takes about 73 seconds.

6. Result

Quantity	Result
Modulation used	Binary FSK (0 = 2000 Hz, 1 = 3200 Hz)
Bit rate	400 bits/s
Image sent	32 x 32 colour (24576 bits, 96 packets)
Transmission time	about 73 s
Bit errors	572
Measured BER	2.33 %
Error-free packets	71 / 96
Received image quality (PSNR)	24.2 dB
Simulated BER at 6 dB Eb/N0	FSK 6.99 %, Hamming(7,4) 3.4 %, Repetition(3) 1.4 %, DBPSK 0.91 %

Thus, the image was transmitted between two smartphones using acoustic Binary FSK and recovered at the receiver with a measured BER of 2.33 %.